Recognising without recalling:

a small recurrent memory recognises long after it stops recalling

Ivan Rodkin

benchmark: `irodkin/recognition-memory`

Takeaway

A memory can tell whether an input is familiar long after it can no longer reproduce it.

- Shrinking a GDN-2 state, **exact recall fails first**: small states still recognise sequences **2–8**× longer than they can copy.
- At 16×4, $N{=}256$ the memory **trained for recognition** is ahead: acc9 **0.88** [0.74–1.00] vs. **0.45** [0.27–0.63] for surprise (chance 0.11).
- Training it needs a **length curriculum**, and at small states it is **not reproducible**: the same seed can learn or collapse.

Hypothesis

Memory need not *reproduce* the past. It can instead *recognise* whether an input is consistent with it, using low **surprise** as the familiarity signal. Checking is easier than generating, so recognition should need **less capacity** than recall.

Benchmark: one sequence, eight near-copies

Uniform i.i.d. tokens, vocabulary 16 (4 bits each), $N = 4 \dots 256$. Each comes with **8 corrupted copies**: one forced change plus $p = \frac{1}{N-1}$ elsewhere, so **2 changes on average**.

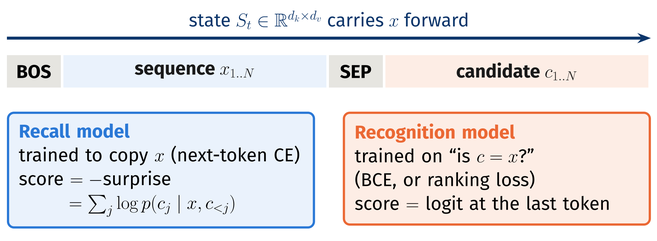
true	3	a	0	f	7	7	c	1	9	e	4	2	b	5	d	8
copy	3	a	0	f	7	2	c	1	9	e	4	2	b	5	d	8

Task. Score the true sequence and its 8 copies.
acc9: true one ranked first (chance $1/9$). **pairwise**: true beats one copy (chance $1/2$).

Capacity, in theory. Recall needs all $4N$ bits (1,024 at $N{=}256$). Recognition needs a few: keeping only the token sum mod 16 (4 bits) gives acc9 0.85; adding a position-weighted sum (8 bits) gives 0.965, at *every* N.

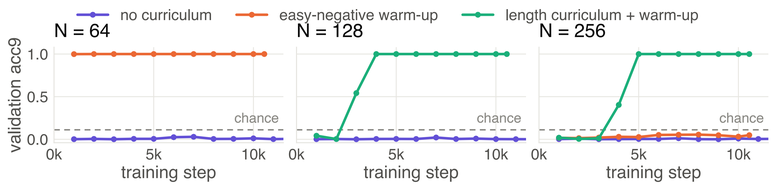
Model: one network, two readouts

2-layer Gated DeltaNet-2 (`fla`), width 128, one head. The only memory is the recurrent state, $d_k{\times}d_v$ per layer: 128×128 (16,384 numbers) down to 16×4 (64).



1 Full-size state: recall is easy, recognition needs a curriculum

At 128×128 the **recall model** copies every length almost perfectly (exact copy 0.988 at $N{=}256$), and its surprise ranks the true sequence first in *every* test sample. The **recognition model**, trained directly, fails from $N{=}64$ on: its loss sits at $\ln 2$ (a constant output) for all 35k steps. The same network learns recall in $<1{,}000$ steps, so this is an **optimisation** failure, not a capacity limit.
Length curriculum: for the first half of training, use prefixes $16, 32, \dots, N$ (equal time each), plus an **easy-negative warm-up** (heavily corrupted negatives that fade out); then full length.

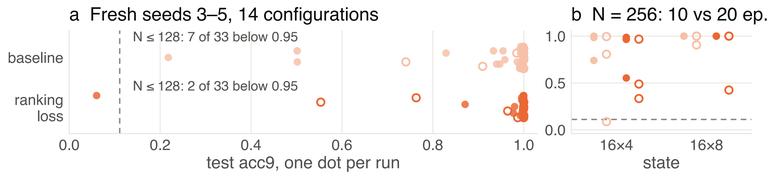


Validation acc9 (full length), 128×128. Curriculum runs: 3 epochs; others: 10 (first 11k steps).

- **Length curriculum**: acc9 $= 1.000$ at $N{=}128$ and 256. The warm-up alone fixes only $N{=}64$.
- With it, **all 72** sweep and seed runs train; none ends at $\ln 2$.

2 Small states: training is a coin flip

The same code and seed gave acc9 **0.871** in the sweep and **0.058** in a rerun on another GPU model (kernels aren't bit-identical). Each small-state run is one draw from a two-humped distribution. Tried: a **ranking loss**, loss $= \mathrm{softplus}(s_{\text{neg}} - s_{\text{true}})$ on the same last-token score, curriculum from length 4.



Recognition model, **baseline** and **ranking loss**, 3 runs per configuration (16×4–16×16). (a) hollow: $N{=}256$. (b) filled: 10 epochs; hollow: 20, more curriculum time on long prefixes.

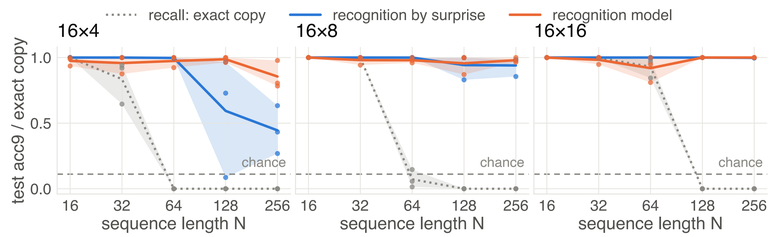
- Up to $N{=}128$ the **ranking loss** fails less (2 vs. 7 of 33 below 0.95; $p{=}0.07$, not significant) but also collapses (0.06); at $N{=}256$ it is worse (0.92 vs. 0.96). 20 epochs hurt both (b).

What a failed run is (28 instrumented reruns):

- **The right comparison, less precise.** It uses order (shuffles caught 0.995) and is no sum checksum (sum-kept edits 0.936 = random 0.937), but catches a single change 0.89 vs. 0.997.
- **Decided during the curriculum:** 78–81% vs. 98–99% single-change detection at its end; only 86–90% after.
- **Small-edit negatives from step 1 don't help** (16×4, $N{=}16, 32$): 7 of 24 runs fail with a 25% share, as without; 11 of 24 with 50%.

3 Shrinking the state: recall goes first

6 state sizes (16×4 to 128×128) × 7 lengths × 2 objectives, 10 epochs each, tested on 100k sequences; 3 seeds at the three smallest states. Recognition model: BCE + length curriculum.



Three smallest states, seeds 0–2, test acc9 and recall: mean (line), min–max (band), each seed (dots).

- **Recognition outlasts recall.** Longest length that passes (powers of 2 tested):

state	16×4	16×8	16×16	32×32	64×64	128×128
exact copy ≥0.9	32	32	64	128	128	≥256
surprise acc9 ≥0.99	64	128	≥256	≥256	≥256	≥256
recognition* acc9 ≥0.99	**128**	≥256	≥256	≥256	≥256	≥256

At the three smallest states recognition reaches **2–8**× the copy length; from 32×32 up, $N{=}256$ hides the gap. At 16×16, $N{=}256$ the recall model keeps ≥536 of 1,024 bits, copies nothing exactly, yet both readouts score ≥0.996. *Best of the 3 sweep runs (1 from 32×32 up); the median run fails at shorter lengths. Fresh seeds 3–5 take 16×4 to ≥256. Bar 0.99: an 8-bit checksum already scores 0.965.

- **Smallest state: the recognition model is ahead.**

16×4	recall tokens	surprise, 3 runs	recognition, 6 runs
$N{=}128$	0.30	0.59 [0.09–0.97]	**0.98** [0.95–1.00]
$N{=}256$	0.20	0.45 [0.27–0.63]	**0.88** [0.74–1.00]

At $N{=}256$ its worst run (0.74) beats surprise's best (0.63). Few runs per cell, but large margins.

- Near capacity, **surprise** depends on what the memory chose to keep: 0.97, 0.73, 0.09 across seeds at $N{=}128$.
- At 16×8 the two are about even ($N{=}256$: 0.94 vs. 0.98).

Next

- **More runs per cell** (6–10, both readouts) to measure failure rates.
- **Restarts**: train several starts for ∼5k steps, continue the lowest-loss one; needs testing, as the step-4k loss predicts the outcome only weakly ($\rho{=}-0.47$). Training changes so far (fixes, ranking loss, early small-edit negatives) don't remove the failures.
- **Harder benchmarks** where checksums fail; K stored sequences per context.

3 seeds at 16×4–16×16 for $N \geq 16$ (6 recognition runs in the 16×4 table), seed 0 elsewhere. The three smallest states shrink d_v only (fla kernels need $d_k \geq 16$); across them the parameter count changes by 4%.